

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore  

Université Côte d’Azur, Inria, France

Enrico Tassi  

Université Côte d’Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the ELPI’s implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λ PROLOG (see [15, 16, 7, 13]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [14, 3, 4, 20, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [18, 19, 12], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [10] to help users tame backtracking. ROCQ users are familiar with functional programming but not necessarily with logic programming, in particular uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker identifies predicates that behave like functions, i.e., predicates that commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [17] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard PROLOG abstract machines [21, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the

sottolineare che
checking = user
da le signature



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:16

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !. % second one is cut away, as well as the choice
g D F :- f D E, f E F.    % points left by the call to r.

```

■ **Figure 1** Running example of a ELPI program

tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1 .

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [10]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. First, we check for overlapping rule heads: if two heads do not overlap in the inputs they accept, then no query can unify with both. Second, we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded and all choice points allocated by premises preceding the cut are discarded. We show that if every rule defining a predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

Both semantics share the same notion of program, written $\mathbb{P}$. Figure 1 is our running example. A program consists of a list of rules $\mathbf{R}$ together with a signature environment sigT . Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, and whether the predicate is deterministic.

A rule consists of a head term Tm and a list of premises. Each premise is an Atom , i.e. either the cut operator or a predicate call. Terms Tm include predicate symbols, data, and variables. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The set of variables $\mathbf{V}$ is countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the `finmap` library [?]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

Definition $\text{bc} : \mathbb{P} \rightarrow \mathcal{F}_V \rightarrow \text{Tm} \rightarrow \Sigma \rightarrow \mathcal{F}_V * \text{list } (\Sigma * \text{list } \mathbf{A}) :=$

<pre> Inductive Tm := ... Pred of P Var of V. Inductive sig := NonDet Det. Definition sigT := P -> sig * nat * nat. </pre>	<pre> Inductive A := cut call : Tm -> A. Record R := { head : Tm; premises : list A }. Record P := { rules : list R; sig : sigT }. Definition Σ := {fmap V -> Tm}. </pre>
---	---

■ **Figure 2** Syntax of ELPI programs

■ **Figure 3** Syntax of ELPI programs

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization we abstract the unifier and its properties, namely

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

TODO: state axioms and explain matching; cite Dale's unification results. In this paper we assume the reader is familiar with first-order unification.

2.1 The cut operator

The cut operator in ELPI implements the *hard cut* (see, e.g., the documentation of SWI-PROLOG [?]). Whenever the backchaining function `bc` returns a list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right. When a cut is encountered, it discards (i) all choice points created by `bc` for the current call and (ii) all choice points created while executing the premises preceding the cut.

For example, consider the program P in Figure 1 and the query $g \ 0 \ R$. Both rules for g apply to this query. Backchaining therefore returns the following two alternatives:

```
[(σ1, [r A B, f B C, !]), (σ2, [f D E, f E F])]
```

with $\sigma_1 := \{A \mapsto 0, B \mapsto C\}$ and $\sigma_2 := \{D \mapsto 0, F \mapsto R\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

Execution continues with the first premise of the first alternative, namely $r \ A \ B$ under σ_1 , i.e. $r \ 0 \ C$. The remaining alternatives are stored as choice points. Backchaining $r \ 0 \ C$ returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, !)]$ where $\sigma_3 := \sigma_1 + \{B \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{B \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{B \mapsto 2\}$.

Since all rules for r have empty premises, we chose the first success, save the others as choice points, and continue with $f \ B \ C$ under σ_3 , that is, $f \ 0 \ C$. This time, backchaining fails (no rule for f matches input 0). We therefore backtrack to the most recently introduced choice point and retry $f \ B \ C$ under σ_4 , i.e. $f \ 1 \ C$. This succeeds with $\sigma_6 := \sigma_4 + \{R \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for r and one from the second rule for g (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for g was selected. (In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

In Sections XX and YY we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

```

Inductive tree :=
  | KO | OK                                (* failure and success *)
  | Todo of Atom                          (* unexplored branch *)
  | Or of option tree &  $\Sigma$  & tree        (*  $A \vee B_\sigma$  *)
  | And of tree & list Atom & tree.        (*  $A \wedge_{B_0} B$  *)

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree}) :=$ 
Definition next_alt :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=$ 
Inductive runT :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow \text{option } (\Sigma * \text{option tree}) \rightarrow \mathbb{B} \rightarrow \mathcal{F}_v \rightarrow \text{Prop} :=$ 

```

■ Figure 4 Signatures of the tree-based semantics

110 2.1.0.1 Notational conventions

111 In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\perp$ for `true` and `false`.
 112 The option instances are noted $\bullet t$ for *Some* t and $\square$ for *None*. Following the Mathematical
 113 Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second
 114 projection applied to the pair `x`.

spiegare nota-
tion for list

115 3 And-Or Tree semantics

TODO: nested
option, output
Fv also inside op-
tion

116 An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that
 117 alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root
 118 (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records
 119 a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving
 120 one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

121 The tree is built incrementally. It therefore contains a `Todo` node, which represents an
 122 unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure
 123 from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer,
 124 and the premises of each applicable rule form the corresponding And-layer. This expansion
 125 step is performed by the `step` procedure described in Section 3.2.

126 The Rocq data type for And-Or trees is given in Section 3. In addition to `Todo`, it provides
 127 two distinguished nodes, `KO` and `OK`, which represent failed and completed branches
 128 (intuitively, an empty disjunction and an empty conjunction, respectively).

129 The *Or* node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing
 130 disjunction A . The left subtree A is optional and is absent once that branch has been fully
 131 explored. The right alternative is paired with a substitution σ , which becomes the current
 132 substitution when backtracking to B .

why option and
not KO?

133 The *And* node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to be solved
 134 after A . We call B_0 the *reset point* for B : it is used to restore the initial state of B when
 135 backtracking occurs within A . An example is shown in Section 3.3.

136 A graphical representation of a tree is shown in Figure 6a. For compactness, we depict
 137 *And* and *Or* nodes as n -ary (rather than binary), with children associating to the right. The
 138 `KO` node acts as the terminating element of an *Or* list, while `OK` is the terminating element
 139 of an *And* list.

```

Fixpoint next  $\sigma$   $t$  :  $\Sigma$  * tree :=
  match  $t$  with
  | Todo _ | KO | OK  $\Rightarrow$  ( $\sigma$ ,  $t$ )
  | Or  $\square$   $\sigma'$  B  $\Rightarrow$  next  $\sigma'$  B
  | Or  $\bullet$ A _  $\Rightarrow$  next  $\sigma$  A
  | And A _ B  $\Rightarrow$ 
    let ( $\sigma'$ , pA) := next  $\sigma$  A in
    if pA == OK then next  $\sigma'$  B
    else ( $\sigma'$ , pA)
  end.

Definition next_subst  $\sigma$   $t$  := (next  $\sigma$   $t$ ).1
Definition next_tree  $t$  := (next  $\varepsilon$   $t$ ).2.

Definition success  $t$  := next_tree  $t$  == OK.
Definition failed  $t$  := next_tree  $t$  == KO.
Definition incomplete  $t$  :=
  if next_tree  $t$  is Todo _ then  $\top$  else  $\perp$ .

```

■ Figure 5 *next* and derived functions

3.1 The tree-based semantics

As anticipated, the tree is constructed incrementally by two recursive functions, *step* and *next_alt*. Both traverse the tree depth-first, left-to-right. Since all functions must terminate in Rocq, we define their transitive closure as the inductive relation *runT*.

Intuitively, *runT* iterates calls to *step* as long as they succeed (tag *Success*). Upon failure (tag *Failed*), it searches for the next alternative and continues from there, if one exists. We detail *step*, *next_alt*, and *runT* in Sections 3.2–3.4.

To support incremental construction, the semantics uses helper functions based on *next*. Given a tree, *next* returns the branch on which the execution should focus, following a depth-first, left-to-right traversal strategy (see Section 3.1).

In all trees in Figure 6, a black arrow marks the result of *next_tree*.

3.2 The *step* procedure

The *step* procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a *tag*, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The *tag* (see Section 3) indicates which kind of step was performed. *Expanded* indicates the expansion of a predicate call via backchain. *Failure* and *Success* are returned for trees that satisfy, respectively, the *failed* and *success* tests. Finally, *CutBrothers* indicates a superficial cut: *step* was called on an Or-layer and the And-layer immediately below it contains a cut.

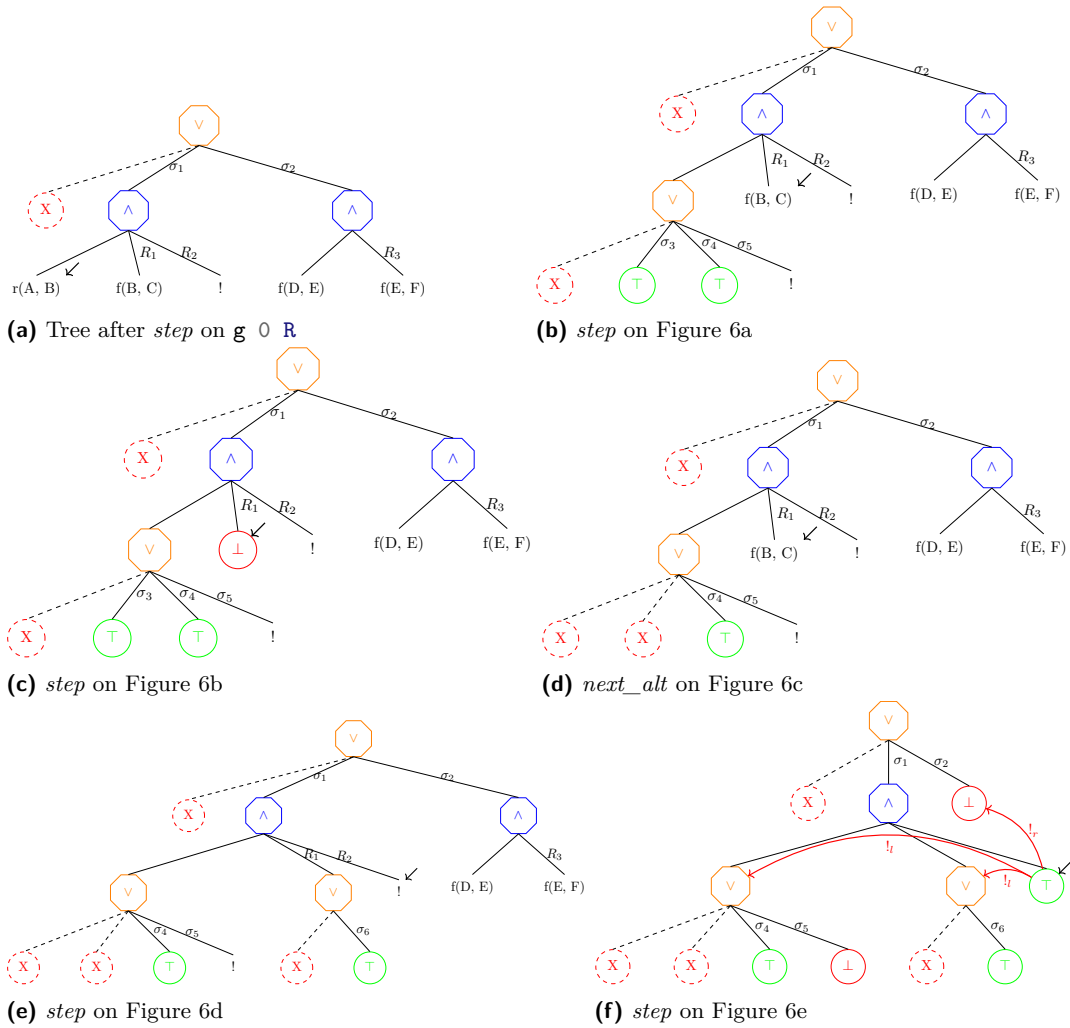
The *step* procedure evaluates atoms. In particular, it leaves the tree unchanged when *next_tree* is false.

Lemma succ_step_iff: $\forall p v \sigma t, \text{success } t \leftrightarrow \text{step } p v \sigma t = (v, \text{Success}, t).$ (1)

Lemma fail_step_iff: $\forall p v \sigma t, \text{failed } t \leftrightarrow \text{step } p v \sigma t = (v, \text{Failed}, t).$ (2)

step expands *Todo* nodes by calling backchain, which returns a list *l* of alternatives. If *l* is empty, then the current call is replaced by *KO*. Otherwise, it is replaced by a right-skewed tree of *Or* nodes, where each leaf corresponds to a list of atoms $e \in l$. If *e* is empty, the leaf is *OK*; otherwise, it is a right-skewed tree of *And* nodes.

¹ The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2 , and R_3 are reset points and correspond respectively to the lists of atoms [f Y Z, !], [!], and [f Y Z].



■ **Figure 6** Some tree representations¹

Figure 6a depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to *step*. We run query $q \ 0 \ R$ against the program P in $??$. Let c_0 , c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P . We need the initial $\square$ subtree so that we can attach a substitution to c_1 . In particular, c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are marked with the R symbol: $R_1 := [f \ Y \ Z, !]$, $R_2 := [!]$, and $R_3 := f \ Y \ Z$. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of *step* triggering backchaining appear in Figures 6b, 6c, and 6e.

sottolineare le
sostituzioni

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by OK ; and (ii) the subtrees in the scope of *Cut* are pruned. More precisely, (ii) prunes the left siblings of the *Cut* node (in the same And-layer, denoted $!_l$) and prunes the right uncles of *Cut* (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 6f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. We note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by KO . This amounts to discarding the third rule of predicate r (see Section 2.1).

3.3 The *next_alt* procedure

When backchain returns an empty list, *step* produces the tag **Failed** and the computation must backtrack. The *next_alt* procedure is responsible for producing a new tree from which the computation can continue.

In Figure 6c we encounter a failure because no rule applies to the atom $f \ X \ Y$ under substitution σ_3 , which maps Y to 0. The *next_alt* procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the OK node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the KO node with the information stored in the reset point R_1 . This restores the call $f \ Y \ Z$, which can then be reconsidered under substitution σ_4 .

Furthermore, *next_alt* serves another important purpose. Its behavior depends on the boolean flag it receives as input: $\text{next_alt} \perp A$ recursively removes all failures (if any) from A , whereas $\text{next_alt} \top A$ removes the first success in A . For example, the tree in Figure 6f contains a successful path that maps R to 2.

Some interesting properties of *next_alt* are shown below; they illustrate how *next_alt* complements *step* with respect to failed, success, and *path_atom* trees.

Lemma *path_atom_next_alt_id*: $\forall b \ t, \text{incomplete } t \rightarrow \text{next_alt } b \ t = \bullet t.$ (3)

Lemma *next_alt_failedF*: $\forall b \ t \ t', \text{next_alt } b \ t = \bullet t' \rightarrow \text{failed } t' = \perp.$ (4)

Lemma *next_altFN_fail*: $\forall t, \text{next_alt } \perp t = \square \rightarrow \text{failed } t.$ (5)

3.4 The *runT* inductive

The inductive relation *runT* relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ , and a tree t . The outputs are (i) an optional result r of the form $\bullet(\sigma', t')$, (ii) a boolean b , and (iii) an updated set of variable names.

211 The main output is r : if $r = \square$, interpretation fails; otherwise σ' is the computed solution
 212 substitution and t' represents the remaining, not-yet-explored alternatives. The boolean b
 213 records whether a superficial cut was evaluated during execution, and the updated variable
 214 set tracks freshly generated names.

215 $runT$ has four cases, depicted as inference rules in Figure 7. (STOPT) If $next_tree\ t$ is a
 216 success, then the substitution σ' is extracted from t (starting from σ) and the input tree is
 217 cleaned of its successful path. (STEPT) If $next_tree\ t$ is an atom, then $step$ is invoked to
 218 evaluate t , and $runT$ is called recursively on the updated tree. (BACKT) If $next_tree\ t$ is a
 219 failure, then $next_alt$ is called to clear the failed path; if the resulting cleaned tree exists,
 220 $runT$ is called recursively on it. Finally, (FAILT) accounts for trees with no solutions.

221 These rules are deterministic:

222 **Lemma runT_det:** $\forall p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',$
 $runT\ p\ v_0\ \sigma\ t\ r\ b\ v \rightarrow runT\ p\ v_0\ \sigma\ t\ r'\ b'\ v' \rightarrow r' = r \wedge v' = v \wedge b' = b.$ (6)

223 The proof is by induction on the first $runT$ derivation and is immediate, because the
 224 rules are selected based on $next_tree\ t$. In particular, the hypotheses $success\ t$, $path_atom\ t$,
 225 and $failed\ t$ are mutually exclusive. Moreover, by Lemma 6, the hypothesis of FAILT implies
 226 $failed\ t$; hence FAILT is exclusive with STOPT and STEPT, and it is also exclusive with
 227 BACKT since they impose different requirements on the output of $next_alt$.

$$\begin{array}{c}
 \frac{success\ t \quad get_subst\ \sigma\ t = \sigma' \quad next_alt\ \top\ t = t'}{runT\ v\ \sigma\ t\ \bullet(\sigma', t') \perp v} \text{STOPT} \\
 \\
 \frac{\frac{path_atom\ t \quad step\ p\ v\ \sigma\ t = (v', tg, t')}{runT\ v\ \sigma\ t\ r\ b'\ v''} \quad \frac{b' = is_cb\ tg\ ||\ b}{runT\ v'\ \sigma\ t'\ r\ b\ v''}}{runT\ v\ \sigma\ t\ r\ b'\ v''} \text{STEPT} \\
 \\
 \frac{failed\ t \quad next_alt\ \perp\ t = \bullet t' \quad runT\ v\ \sigma\ t'\ r\ n\ v'}{runT\ v\ \sigma\ t\ r\ n\ v'} \text{BACKT} \\
 \\
 \frac{next_alt\ \perp\ t = \square}{runT\ v\ \sigma\ t\ \square \perp v} \text{FAILT}
 \end{array}$$

■ **Figure 7** Rule system for $runT$

228 3.5 Properties of $runT$

229 Working with $runT$ allows to prove some interesting properties about the behavior of the
 230 cut operator wrt disjunction, i.e. the Or node. We have proven several properties that are
 231 available at [this link](#). Below we show two of these lemmas.

232 **Lemma runSST_or:** $\forall p\ v\ v'\ \sigma\ \sigma'\ l\ l'\ \sigma_1\ r,$
 $runT\ p\ v\ \sigma\ l\ \bullet(\sigma', \bullet l') \top v' \rightarrow$
 $runT\ p\ v\ \sigma\ (Or\ \bullet l\ \sigma_1\ r)\ \bullet(\sigma', \bullet(Or\ \bullet l'\ \sigma_1\ KO)) \perp v'.$ (7)

233 **Lemma runNT_or:** $\forall p\ v\ v'\ \sigma\ t\ \sigma_1\ t',$
 $runT\ p\ v\ \sigma\ t\ \square \top v' \rightarrow$
 $runT\ p\ v\ \sigma\ (Or\ \bullet t\ \sigma_1\ t')\ \square \perp v'.$ (8)

234 **Lemma run_orSST** $p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ l\ l'\ r\ r':$
 $runT\ p\ v\ \sigma\ (Or\ \bullet l\ \sigma_1\ r)\ \bullet(\sigma', \bullet(Or\ \bullet l'\ \sigma_1\ r')) \perp v' \rightarrow$
 $\exists b, runT\ p\ v\ \sigma\ l\ \bullet(\sigma', \bullet l')\ b\ v' \wedge r' = \text{if } b \text{ then } KO \text{ else } r.$ (9)

Definition $\text{stepE } v \ t \ \sigma \ a \ g :=$
 $\text{let } (v', r) := \text{bc p } v \ t \ \sigma \ \text{in}$
 $(v', \text{save_alts } a \ g \ r).$

$$\frac{}{\text{runE } v \ ((\sigma, \emptyset) :: a) \bullet(\sigma, a)} \text{STOPE} \qquad \frac{\text{runE } v \ ((\sigma, g) :: ca) \ r}{\text{runE } v \ ((\sigma, (\text{cut}, ca) :: g) :: _) \ r} \text{CUTE}$$

$$\frac{\text{stepE } v \ t \ \sigma \ a \ g = (v_1, a') \quad \text{runE } v_1 \ (a' ++ a) \ r}{\text{runE } v \ ((\sigma, (\text{call } t, _) :: g) :: a) \ r} \text{CALLE} \qquad \frac{}{\text{runE } _ \ \emptyset \ \square} \text{FAILE}$$

■ **Figure 8** Rule system for runE

235 In Lemma 7, we evaluate the tree l . This evaluation produces a success with signature σ'
 236 and a new tree l' . During the evaluation, a superficial cut in l is traversed. This implies that
 237 the evaluation of $\bullet l \vee_{\sigma_1} r$ returns σ' as substitution, and the alternatives are $\bullet l' \vee_{\sigma_1} \text{KO}$. The
 238 right-hand side of the Or node is replaced with KO by $!_r$.

239 In Lemma 8, we are in a similar scenario: t evaluates a superficial cut but, this time,
 240 produces a failure. This implies that the evaluation of $\bullet t \vee_{\sigma_1} t'$ also fails.

241 Finally, in Lemma 9, we evaluate $\bullet l \vee_{\sigma_1} r$. The evaluation succeeds, producing the result
 242 $\bullet(s_1, \bullet l' \vee_{\sigma_1} r')$. This means that l still has l' as unexplored alternatives. We deduce that
 243 the evaluation of l produces the substitution σ_1 with alternatives l' . However, we have no
 244 information about whether a superficial cut has been crossed in l . If this is the case, we
 245 know that r' is cut away by $!_r$; otherwise, r' has not been explored and is equal to r .

246 From these lemmas, it is clear how the order in which alternatives are evaluated changes
 247 the behavior of the program.

248 4 Stack semantics

249 We now introduce the ELPI semantics. The interpreter is close to that of the ELPI language
 250 and provides an operational semantics that is similar to the one proposed by Pusch in [17],
 251 which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3].
 252 Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that
 253 are also present in the Warren Abstract Machine [21, 1].

254 We represent a state of the ELPI language thanks to the two mutual inductives shown
 255 below.

Inductive $\text{alts} := \text{nilA} \mid \text{consA of } (\Sigma * \text{goals}) \ \& \ \text{alts}$
with $\text{goals} := \text{nilG} \mid \text{consG of } (A * \text{alts}) \ \& \ \text{goals}.$

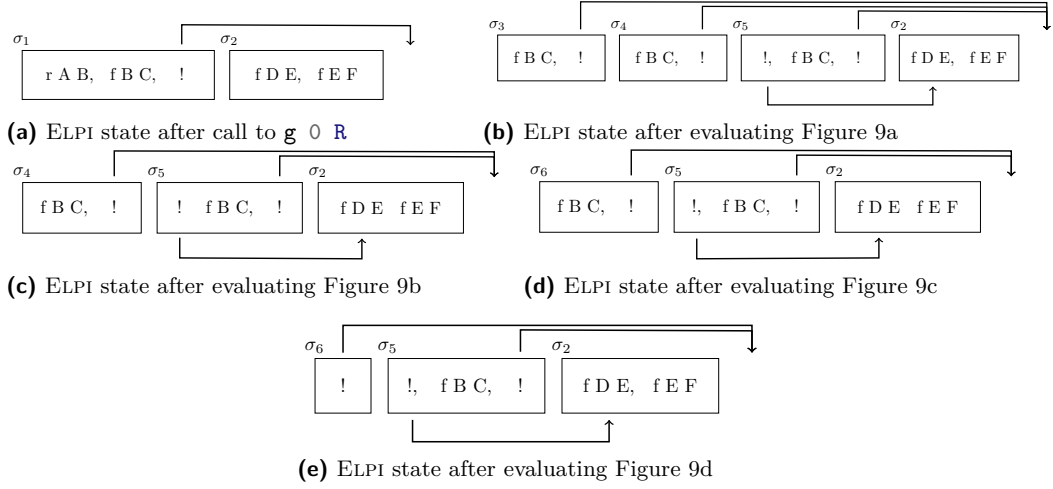
256 An ELPI state is an enhanced two-dimensional list. The outermost list represents a list
 257 of alternatives in disjunction, each accompanied by the substitution that should be used
 258 for their interpretation. The innermost list is a list of atoms, that is, the list of goals in
 259 conjunction. These goals are decorated with a pointer to an alternative list, which is used to
 260 keep track of where to jump when a cut is interpreted. We call these special alternatives the
 261 *cut-to* alternatives.

262 The ELPI interpreter is called runE and has the following type.

263 **Inductive** $\text{runS} : \mathcal{F}_v \rightarrow \text{alts} \rightarrow \text{option } (\Sigma * \text{alts}) \rightarrow \text{Prop} :=$ (1)

264 The inductive runE represents a routine taking as input a set of variables and a list
 265 of alternatives and returning an option. $\square$ stands for interpretation with no solution,

se riesco success
 (A or B) = suc-
 cess (B or A) if
 no sup cut in A
 and B



■ **Figure 9** Example of ELPI execution

whereas $\bullet(\sigma, a)$ represent a successful interpretation with σ as solution and a as the list of non-yet-explored choice points.

The rule system for *runE* is given in Figure 8 and is made by 4 cases. We distinguish two main cases: if the list of alternatives is empty, (FAILE) we have a failure: we stop the computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter evaluates the first choice point h under the substitution σ_0 . If h is empty, (STOPE), then we have a success, i.e. we return the input σ_0 and leave a as the future choice points. Otherwise, h is the list $(t, ca) :: g$ where t is the first atom to evaluate, ca is its cut-alternative list and g is the list of future conjuncts. If t is a cut (CUTE), we need to keep evaluating g under σ_0 but we change the alternatives to ca . Finally, in the case of a call (CALLE), we backchain the call in the program. If rs is the list of rule premises returned by *bc*, then each list of premises if mapped to a list of goals (goals) have a as cut-alternative and the global list is translated to a list of alternative (alts).

Example of execution We propose an exemple of execution of an ELPI state in Figure 9. The arrows in the images represent the cut-alternative pointer of the cut. We have not put arrow going our from atom-calls since the interpreter ignores the cut-alternatives in this case (cf. CALLE).

Remark We want finally to point out how the lemma that we were able to prove in Section 3.5 are not easy to be expressed in the ELPI semantics. This is beacuse there is no sufficient structure in the ELPI state to understand what is the scope of the cut operator. For example, in an ELPI state made of these alternatives $a_0, a_1 \dots a_n$, we have no clear idea of what happens if the first alternative a_0 succeeds: there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives $a_1 \dots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

5 Equivalence of the two semantics

In the previous section we have introduces the two semantics, they are The equivalence between the two semantics is possible under two conditions: we need to work with “valid tree”, i.e. all of those trees that can be generated from a call. Secondly, we need to translate

```

295 Fixpoint valid_tree A :=
296   match A with
297   | Todo _ | OK | KO  $\Rightarrow$   $\top$ 
298   | Or  $\square$  _ B  $\Rightarrow$  valid_tree B
299   | Or  $\bullet$ A _ B  $\Rightarrow$  valid_tree A &&
300     ((B == KO) || B.base_or B)
301   | And A B0 B  $\Rightarrow$  valid_tree A &&
302     if success A then valid_tree B
303     else B == big_and B0
304   end.
305
306 Fixpoint t2l t  $\sigma$  (cp: alts) : alts :=
307   match t with
308   | OK  $\Rightarrow$  [( $\sigma$ ,  $\emptyset$ )]
309   | KO  $\Rightarrow$   $\emptyset$ 
310   | TA a  $\Rightarrow$  [( $\sigma$ , [(a,  $\emptyset$ )])]
311   ...
312
313 (a) Valid tree
314
315 (b) Tree to list

```

■ **Figure 10** Valid tree and Tree to list

295 trees into elpi states. In the next to subsection we propose the two functions *valid_state* and
 296 *t2l*.

297 5.1 Valid trees (*valid_state*)

298 The inductive tree allows one to generate a large number of trees, some of which are not
 299 valid, in the sense that they cannot be produced starting from a given query. The class of
 300 valid trees is characterized by the function shown in Figure 10a.

301 While all base cases of tree are considered valid, we need to analyze carefully the cases
 302 for the *Or* and *And* constructors.

303 For the *Or* constructor, we distinguish two cases depending on whether the lhs exists. If
 304 it does not exist, then the rhs must be a valid tree. Otherwise, the lhs must itself be a valid
 305 tree, and the rhs is either the *KO* tree, since it may have been removed by the evaluation of
 306 a superficial cut in the lhs, or it has not yet been explored. In the latter case, it is a *base_or*
 307 tree, namely the right-skewed tree formed by a disjunction of conjunctions that is generated
 308 after a successful backchain to a call.

309 For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs
 310 depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never
 311 been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds.
 312 In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms,
 313 the reset point B_0 ensure what shape the rhs should have. If the lhs is a success tree, then
 314 the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

315 5.2 From trees to lists (*t2l*)

316 The translation of a tree to a en ELPI state is done thanks to the *t2l* function. In Figure 10b
 317 we give just the implementation of the base cases, the full implementation is available at
 318 [this link](#).

319 The function *t2l* takes the tree t_0 we want to translate, a substitution σ_0 and a list of
 320 choice points *cp*. These choice points represent alternatives that appears at the same level
 321 of the current node, such as, for example, the right-siblings of a *Or* node. The *OK* node
 322 represent one successful alternative, therefore it is translated into a singleton list with the
 323 empty list of goals. The *KO* node represent a failure, i.e. it is mapped to the empty list of
 324 alternatives. Finally, atoms are mapped to one alternative containing a single goal with the
 325 empty cut-alternative list. They cannot point to *cp* since they are not right-uncle subtrees.

t	6a	6b	6c and 6d	6e	6f
$t2l\ t \in \emptyset$	9a	9b	9c	9d	9e

■ **Table 1** Tree to list from Figure 6 and Figure 9

We have omitted the code for the translation of the *Or* and *And* constructor since it is quite complex, but, we give the intuition behind this translation by means of examples.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B . The alternatives of the rhs are valid choice points for the lhs and should be passed to the translation of A . The two list of alternatives can be concatenated and, each atom, should add cp as new cut-to alternatives: A and B are one level lower wrt cp and therefore the cut they have should point to cp .

The translation of $A \wedge_{B_0} B$ implies the translation of A . If the translation of A returns an empty list we return the empty list of alternatives without even touching B nor B_0 . Otherwise the translation returns a list $(\sigma_0, g) :: a$. By the semantics of *runT* the B node represent the continuation of the first alternatives in A , i.e. g , whereas B_0 is the continuation of goals for each alternative in a . While performing these translation, we need to carefully update the cut-alternatives.

Example of t2l In Table 1 we point out the result of calling *t2l* on the trees in Figure 6. The first row contains the image reference of then trees and the second row contains the reference of the images from the ELPI state in Figure 9. For the 3rd column in the table, we see that *t2l* is not injective. There are different trees that maps to the same ELPI state. In fact, there are transitions in *runT* that are not visible in *runE*.

5.3 Equivalence theorems

Thanks to the two functions *valid_state* and *t2l*, we are able to finally state the equivalence theorems between the semantics. The proof we are giving are on a slightly different definition of *runT*, since in the equivalence, we ignore the verification for the boolean and the variable set returned by *runT*, these piece of data are particularly meant for the proof in Section 3.5.

The definition we use is the following:

Definition *runT'* $p\ v\ \sigma\ t\ r := \exists\ b\ v',\ \text{runT}\ p\ v\ \sigma\ t\ r\ b\ v'.$ (2)

Before going to the equivalence proofs, we want to claim a key lemma allowing to ignore the silent transition in the *runT* inductive.

Lemma *next_altF_t2l*: $\forall\ t\ t'\ b,$
 $\text{valid_tree}\ t \rightarrow \text{failed}\ t \rightarrow \text{next_alt}\ b\ t = \bullet t' \rightarrow$
 $\forall\ l\ \sigma,\ \text{t2l}\ t\ \sigma\ l = \text{t2l}\ t'\ \sigma\ l.$ (10)

It is proved easily by induction on the tree t .

Lemma *tree_to_elpi*: $\forall\ p\ t\ \sigma\ r,$
 $\text{let}\ v := \text{vars_tree}\ t \mid \text{vars_sigma}\ \sigma\ \text{in}$
 $\text{valid_tree}\ t \rightarrow$
 $\text{runT}'\ p\ v\ \sigma\ t\ r \rightarrow$
 $\text{let}\ a := \text{t2l}\ t\ \sigma\ \emptyset\ \text{in}$
 $\text{let}\ r' := \text{if}\ r\ \text{is}\ \bullet(\sigma',\ t)\ \text{then}\ \bullet(\sigma',\ \text{t2l}\ (\text{odflt}\ K0\ t)\ \sigma\ \emptyset)$
 $\text{else}\ \square\ \text{in}$
 $\text{runS}\ p\ v\ a\ r'.$ (11)

Proof sketch We proceed by induction on the execution of *runT'*. There are 4 cases to be treated. The first case comes from *STOPT*, it deals with successful trees, a successful tree must to start with an empty list and therefore we conclude with *STOPE*. The case for

359 STEPT, need to treat two cases: *step* has evaluated either a cut or a call. Depending on this
 360 we should use CUTE or CALLE and then the induction hypothesis. The case for BACKT is
 361 a silent case: it represents the non-injective behavior of *t2l*, but, thanks to Lemma 10 and
 362 the induction hypotheses the conclusion is proved. The last case comes from the derivation
 363 FAILT. It is easily proved using FAILE and the fact that if *next_alt* returns $\square$ when trying
 364 to ease failure in *t*, then *t2l* of *t* returns the empty list.

```

365 Lemma elpi_to_tree p v a r :
    runS p v a r  $\rightarrow$ 
     $\forall \sigma t, \text{valid\_tree } t \rightarrow \text{t2l } t \sigma \emptyset = a \rightarrow$ 
    if r is  $\bullet(\sigma', a')$  then
       $\exists t', \text{runT}' p v \sigma t \bullet(\sigma', t') \wedge \text{t2l } (\text{odflt } K0 \ t') \sigma \emptyset = a'$ 
    else runT' p v  $\sigma t \square$ .
  
```

(12)

366 The proof of Lemma 12 is based on the idea explained in [2, Section
 367 3.5.1]. An ideal statement for this lemma would be: given a function
 368 *l2t* transforming an ELPI state *a* to a tree *t*, we then would be able
 369 to compare the result of *runE* run with *a* and *runT'* run with *t*. The
 370 function it is difficult to implement since we have hard time finding the
 371 tree structure of *a*, and more than that, we should have a notation of
 372 ELPI valid state, but again, this is a hard task. We therefore need to
 373 reuse the *t2l* function to relate a tree *t* with *a*. Figure 11 show the idea
 374 of the proof: we have a state *t* whose translation is *a*, we proceed by
 375 induction on the execution of *runE*. This gives us not only the output
 376 *a'* but that hypothesis that it exists a tree *t'* such that its translation
 377 to the ELPI state is *a'*. This proof strategy is indirectly providing a kind of *l2t* function by
 378 combining the execution of the *runE* and *t2l*.

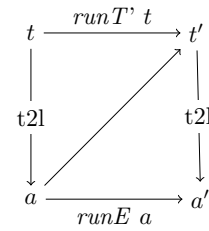


Figure 11

6 Case study: determinacy analysis

380 An interesting application of the tree semantics is the determinacy checking that was added
 381 in version 3 of the ELPI language [11]. The checker is meant to receive the signature of the
 382 predicates declared by the user and then verify that the rules of a deterministic predicate
 383 will return at most one solution, we define this particular kind of predicate as functions.
 384 Thanks to this static verification, the user can better control unexpected backtracking at
 385 compile time: if an incoherence is rilevata then a fatal error explain why the current program
 386 may break determinacy. A function behaves deterministically if two main conditions are
 387 respected: *mutual exclusion* guarantees that at most one rule can be successfully used on a
 388 given query, whereas *rule checking* ensure that each rule does not creates choice points, e.g.
 389 by calling non-deterministic predicates.

For example, in ??, we can declare the following signatures to the predicates.

```

f : int  $\rightarrow_i$  int  $\rightarrow_o$  func    g : int  $\rightarrow_i$  int  $\rightarrow_o$  func    r : int  $\rightarrow_i$  int  $\rightarrow_o$  pred
  
```

390 We see that **f** is a function since its rules are mutual exclusive and have empty body and
 391 **g** is a function since **r1** is cutting **r2** if **r1** succeeds due to the cut. Moreover, both rules

392 The signature of the program are stored in the its **sig** projection, and, as in [11] we need
 393 to extends the type of predicates so that we are able to distinguish between deterministic
 394 and non-deterministic propositions. For example, in ??, we have ...

395 A program execution behaves deterministically if given a program, a substitution and a
 396 tree to the *runT* inductive, then either the execution fails, or, if it succeeds, then the tree of
 397 alternatives is $\square$. Below we define **is_det**.

spiegare sintassi

ref verso pro-
gramma

Definition `is_det` $p \sigma v t :=$
 $\forall r, \text{runT}' p v \sigma t r \rightarrow \text{if } r \text{ is } \bullet(_, x) \text{ then } x = \square \text{ else } r = \square.$

use coq syntax
instead of elpi?

The theorem we want to prove is the following.

Lemma `det_check_call` $p \sigma t:$
`let` $v := \text{vars_tm } t \text{ `|` vars_sigma } \sigma$ `in`
`check_` $\mathbb{P} p \rightarrow \text{tm_is_det } p.(\text{sig}) t \rightarrow \text{is_det } p \sigma v (\text{Todo } (\text{call } t)).$

399 With `check_` $\mathbb{P}$ being the function checking the program wrt mutual exclusion and rule
 400 checking and `tm_is_det` being the function testing if the term head refers to a rigid constant
 401 with deterministically type. The proof of this lemma should be done on the derivation of the
 402 program execution. However, in this definition, the induction hypothesis is too weak, since
 403 in we would have to prove ... but we will not be able to show that the ... is deterministic.

404 To solve the problem, we need to work on “deterministic trees” (`det_tree`), show that if
 405 a tree respect this condition, and then prove the following theorem which is more generic
 406 then `det_check_call`.

Lemma `det_check_tree` $\sigma v p t:$
`vars_tree` $t \text{ `<=` } v \rightarrow \text{vars_sigma } \sigma \text{ `<=` } v \rightarrow$
`check_` $\mathbb{P} p \rightarrow \text{det_tree } p.(\text{sig}) t \rightarrow \text{is_det } p \sigma v t.$

407 Since `det_check_call` is an instance of `det_check_tree`, its proof is now trivial.

408 As an important remark, proving the same lemma with the stack semantics, it is difficult
 409 to correctly define the predicate `det_stack`, checking for the deterministic behavior of a generic
 410 stack, since, similar to Section 3.5, the lack of structure make this goal hard. We need
 411 therefore an intermediate data structure, like the tree to encompass this problem.

412 7 Related work

413 The input mode of Elpi, inspired by the XXXX of BProlog, has only an impact in the
 414 determinacy analysis that does not required to be preceeded by a mode analysis. Well moded
 415 program propagate the groundness from input arguments to output arguments. As a result
 416 if the initial query is ground then unification degenerates to matching (for input arguments).
 417 Considering a semantics where input arguments are match has the following impact on det
 418 analysis: axiom XXX does not require q to be ground.

419 The only mechanization of Prolog’s semantics we could find is the one by Pusch in
 420 ISABELLE/HOL [17] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their
 421 work start from that semantics and refines it by introducing some optimizations usually
 422 found in the Warren abstract machine [21, 1]. They do not use the semantics to prove
 423 properties on the execution of programs as we do in our use case, hence they do not face the
 424 difficulty described in seciton XXX that pushed us to develop an more explicit semantics
 425 (with respect to cut). In some way our work is complementary, showing he stack based
 426 semantics equivalent to more optimized one, we could consider implement in Elpi.

427 Another relevant work are the ones by King cite, but we could not find their code. Their
 428 semantics is denotational and cannot easily cover all the features our paper XX does, but
 429 would have been sufficient for this first step.

430 The proof technique we use to relate the two semantics is inspired by XXX and we thank
 431 Y.Bertot for insightful discussions on the topic. In XXX Bertot relates bla bla, witout never
 432 constructing a decompiler, exactly as we never build a list to tree function.

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

- 1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34>, doi:10.4230/LIPIcs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.
- 7 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.
- 8 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 9 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- 10 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.
- 11 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- 12 Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
- 13 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

- 482 **14** Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
 483 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
 484 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-*
 485 *niz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
 486 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27)
 487 [de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 488 **15** Dale Miller. A logic programming language with lambda-abstraction, function variables, and
 489 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 490 **16** Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
 491 University Press, 2012.
- 492 **17** Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
 493 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
 494 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
 495 Berlin Heidelberg.
- 496 **18** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
 497 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
 498 2018. URL: <https://inria.hal.science/hal-01637063>.
- 499 **19** Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
 500 *LIPIcs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
 501 doi:10.4230/LIPIcs.CVIT.2016.23.
- 502 **20** Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
 503 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 504 **21** David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
 505 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
 506 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
 507 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).